Supplementary Material

Extended Experimental Protocol for the Grover–Rudolph Hardware Characterisation Campaign on the SpinQ Triangulum Platform

Antonio Falcó, Daniela Falcó-Pomares, Juan-Carlos Latorre, Yuefeng Feng, *et al.*

Companion to: "Practical Grover–Rudolph state preparation on a 3-qubit NMR quantum computer: from rigorous ancilla-free compilation to staged experimental diagnostics"

Version 2.0 — March 31, 2026

Abstract

This document provides the complete experimental protocol for the extended Grover–Rudolph characterisation campaign on the SpinQ Triangulum 3-qubit NMR quantum computer. The campaign extends the first study (Protocol v1.0) in one central direction: rather than characterising hardware performance for a single benchmark distribution, it systematically probes the full state-preparation capability of the device across a heterogeneous suite of seven target probability distributions covering the full range of Shannon entropy, symmetry, concentration, and structural complexity that the 3-qubit Grover–Rudolph construction can address. This broader characterisation allows for a distribution-dependent performance map of the Triangulum, separates hardware-generic error from circuit-structure-specific error, and provides a robust statistical baseline for cross-platform comparison. The campaign is designed for automated execution: a parametric pipeline generates Grover–Rudolph angles, compiles circuits, submits jobs, and aggregates results for each target distribution without manual intervention between runs. The protocol is organised into seven sections: (§1) prerequisites and hardware setup; (§2) the target distribution suite; (§3) the parametric Grover–Rudolph compilation engine; (§4) circuit structure and ladder variants; (§5) automated execution pipeline; (§6) data normalization, readout calibration, and metric computation; (§7) cross-distribution analysis and characterisation outputs.

Contents

1 Prerequisites and hardware setup **2**
1.1 Hardware platform . . . 2
1.2 Software dependencies . . . 2
1.3 Qubit register conventions . . . 3

2 Target distribution suite **3**
2.1 Design rationale . . . 3
2.2 The seven target distributions . . . 3
2.3 Rationale for each distribution . . . 4
2.4 Generating the random distributions . . . 4

3 Parametric Grover–Rudolph compilation engine **5**
3.1 General angle formula . . . 5
3.2 Walsh–Hadamard transform for the UCRy block . . . 5
3.3 Reference implementation . . . 5

4 Circuit structure and ladder variants **6**
4.1 Diagnostic stages . . . 6
4.2 Ladder variants A and B . . . 7

5 Automated execution pipeline **7**
5.1 Campaign structure . . . 7
5.2 Pipeline architecture . . . 7
5.3 Orchestrator . . . 8
5.4 Execution order and drift mitigation . . . 8
5.5 Run log schema . . . 9

6 Data normalization, readout calibration, and metrics **9**
6.1 Probability normalization and marginals . . . 9
6.2 Readout calibration . . . 9
6.3 Run-level metrics . . . 10
6.4 Output data artifacts . . . 10

7 Cross-distribution analysis and characterisation outputs **10**
7.1 Primary characterisation plot . . . 10
7.2 Stage-wise degradation per distribution . . . 10
7.3 Ladder A vs. B comparison per distribution . . . 11
7.4 Readout error characterisation across distributions . . . 11
7.5 Regression: fidelity vs. circuit properties . . . 11

A Distribution checksums **12**

B Expected results: first campaign anchor (D1) **12**

1 Prerequisites and hardware setup

1.1 Hardware platform

The experiments are carried out on the **SpinQ Triangulum**, a commercial 3-qubit desktop NMR quantum computer [3]. The device exposes the native gate dictionary $\{R_y(\theta), X, \mathrm{CNOT}\}$. All higher-level operations are decomposed into this set prior to execution. The campaign has been designed to run on one or two Triangulum units simultaneously; if two units are available, the distribution suite can be split across devices to reduce total wall-clock time (see Section 5).

Required hardware. One or two SpinQ Triangulum units with firmware version recorded in every run log.

Pre-session check. Before each session, verify that a bare measurement (no gates) returns $|000\rangle$ with probability ≥ 0.98. If this threshold is not met, allow ≥ 30 minutes of thermal equilibration and repeat.

1.2 Software dependencies

- `numpy` ≥ 1.24
- `scipy` ≥ 1.10
- `pandas` ≥ 2.0
- `jsonlines` ≥ 3.1
- SpinQ SDK (version as shipped with the unit; record in log)

Full implementation:

https://github.com/afalco/grover-rudolph-practical-implementation

1.3 Qubit register conventions

The register is (q_0, q_1, q_2) with q_0 as the most significant bit:

$$|b_0 b_1 b_2\rangle \leftrightarrow k = 4b_0 + 2b_1 + b_2, \quad b_i \in \{0, 1\},$$

giving ordered basis $\{|000\rangle, |001\rangle, |010\rangle, |011\rangle, |100\rangle, |101\rangle, |110\rangle, |111\rangle\}$, indexed $k = 0, \ldots, 7$.

2 Target distribution suite

2.1 Design rationale

The first campaign [4] used a single symmetric benchmark distribution (D1 below). While informative, a single target cannot distinguish between hardware-generic degradation and circuit-structure-specific error: a device may perform well on symmetric, low-contrast targets yet poorly on concentrated or asymmetric ones. The extended campaign addresses this by probing seven target distributions that span the relevant axes of variation:

- **Shannon entropy** $H_S(p) = -\sum_k p_k \log_2 p_k \in [0, 3]$: from maximally concentrated (near 0) to uniform (3 bits).
- **Symmetry:** symmetric vs. monotonic vs. bimodal vs. unstructured.
- **Contrast:** ratio of largest to smallest nonzero probability, ranging from 1 (uniform) to $\gg 1$ (concentrated).
- **Support size:** all 8 states active vs. effective support on a subset.

No two distributions in the suite are close in all three axes simultaneously, ensuring that the hardware is genuinely exercised in qualitatively different regimes.

2.2 The seven target distributions

Table 1 defines the seven distributions. Each has been chosen so that its Grover–Rudolph angles are non-degenerate (no angle collapses to 0° or 180° except intentionally), and so that the suite as a whole covers the entropy range $[0.54, 3.00]$ bits with no large gaps.

Table 1: Target distribution suite. H_S: Shannon entropy in bits. Contrast: $\max_k p_k / \min_{k: p_k > 0} p_k$. Distributions D5 and D6 are generated from fixed random seeds (see §2.4) and their values are approximate.

ID	Type	$p = (p_0, \ldots, p_7)$	H_S (bits)	Contrast
D1	Symmetric (reference)	$(0.05, 0.10, 0.15, 0.20, 0.20, 0.10, 0.15, 0.05)$	2.77	4.0
D0	Uniform	$(0.125, 0.125, 0.125, 0.125, 0.125, 0.125, 0.125, 0.125)$	3.00	1.0
D2	Monotone decreasing	$(0.35, 0.25, 0.15, 0.10, 0.07, 0.04, 0.03, 0.01)$	2.37	35.0
D3	Bimodal	$(0.02, 0.03, 0.05, 0.40, 0.40, 0.05, 0.03, 0.02)$	2.17	20.0
D4	Unimodal / bell	$(0.02, 0.05, 0.12, 0.31, 0.31, 0.12, 0.05, 0.02)$	2.53	15.5
D5	Random (sparse)	Dirichlet$(\alpha = 0.5)$, seed 42	≈ 1.8	$\gg 1$
D6	Random (generic)	Dirichlet$(\alpha = 1.0)$, seed 137	≈ 2.5	~ 10

Note. Distributions D5 and D6 (shaded blue) are generated programmatically from fixed seeds (see §2.4) and must be computed once before the campaign begins. Their exact values must be stored in the campaign configuration file so that every participant uses identical targets.

2.3 Rationale for each distribution

D0 — Uniform. The uniform distribution is the trivial fixed point of the Grover–Rudolph tree: all splitting angles equal 90° and the UCRy block reduces to a product of single-qubit Hadamard-like rotations. It provides an absolute baseline and tests whether the hardware can maintain equal amplitudes in the presence of crosstalk and relaxation.

D1 — Symmetric (reference). The distribution used in the first campaign [4]. Retained as the inter-campaign anchor point. Any deviation from the first campaign's results in this distribution can be attributed to hardware drift rather than experimental design differences.

D2 — Monotone decreasing. Strong asymmetry at every level of the dyadic tree: the root angle ($\theta_0 \approx 53°$) departs significantly from 90°, and the level-1 and level-2 angles are likewise unbalanced. This stresses the UCRy block with highly non-uniform control rotations and exercises the hardware in a regime qualitatively different from D1.

D3 — Bimodal. Mass concentrated near two internal states ($|011\rangle$ and $|100\rangle$). The root split is nearly balanced ($\theta_0 \approx 90°$), but the level-2 UCRy block must produce very small rotations ($\theta_{2|00}, \theta_{2|11} \to 0$) alongside large ones ($\theta_{2|01}, \theta_{2|10} \to 90°$). This tests whether the hardware can faithfully execute near-zero controlled rotations without crosstalk-induced leakage.

D4 — Unimodal / bell. A unimodal distribution that is close to D1 in entropy but has a higher contrast and a more pronounced central peak. It serves as an intermediate between the symmetric reference and the bimodal case, and tests the device's ability to suppress tails.

D5 — Random sparse (Dirichlet $\alpha = 0.5$). A randomly drawn distribution with concentration parameter below 1, which tends to produce sparse vectors with most mass on a few states. It has no exploitable symmetry and produces generically unbalanced trees. The low-α regime is relevant for practical amplitude encoding scenarios where the target law is irregular.

D6 — Random generic (Dirichlet $\alpha = 1.0$). A randomly drawn distribution from the uniform prior over the probability simplex. It serves as a generic test case without engineered structure. Together with D5, it assesses whether the performance trends observed in D0–D4 generalize to arbitrary targets.

2.4 Generating the random distributions

Distributions D5 and D6 are generated as follows and must be computed once, stored, and shared with all participants before the campaign:

Listing 1: Deterministic generation of random distributions D5 and D6.

```python
import numpy as np

def dirichlet_dist(alpha: float, seed: int,
                   n: int = 8) -> np.ndarray:
    rng = np.random.default_rng(seed)
    p = rng.dirichlet(np.full(n, alpha))
    # Clip any value below 1e-4 to avoid near-zero entries
    p = np.clip(p, 1e-4, None)
    return p / p.sum()

D5 = dirichlet_dist(alpha=0.5, seed=42)
D6 = dirichlet_dist(alpha=1.0, seed=137)
```

```python

# Persist to campaign config
import json
config = {"D5": D5.tolist(), "D6": D6.tolist()}
with open("campaign_distributions.json", "w") as f:
    json.dump(config, f, indent=2)
```

Mandatory verification. Before submitting any circuit to hardware, all participants must load `campaign_distributions.json` from the shared repository and verify that their local values of D5 and D6 match to at least 12 decimal places. Use the reference checksums provided in Appendix A.

3 Parametric Grover–Rudolph compilation engine

3.1 General angle formula

For a general 3-qubit target $p = (p_0, \ldots, p_7)$, the Grover–Rudolph rotation angles are obtained from the dyadic conditional masses [1, 2]. Define the partial sums:

$$s_L = p_0 + p_1 + p_2 + p_3, \quad s_R = p_4 + p_5 + p_6 + p_7, \tag{1}$$

$$s_{LL} = p_0 + p_1, \quad s_{LR} = p_2 + p_3, \quad s_{RL} = p_4 + p_5, \quad s_{RR} = p_6 + p_7. \tag{2}$$

The rotation angles are then:

$$\theta_0 = 2 \arccos\sqrt{s_L}, \tag{3}$$

$$\theta_{1|0} = 2 \arccos\sqrt{s_{LL}/s_L}, \qquad \theta_{1|1} = 2 \arccos\sqrt{s_{RL}/s_R}, \tag{4}$$

$$\theta_{2|b_0 b_1} = 2 \arccos\sqrt{p_{b_0 b_1 0}/(p_{b_0 b_1 0} + p_{b_0 b_1 1})}, \quad (b_0, b_1) \in \{00, 01, 10, 11\}, \tag{5}$$

where $p_{b_0 b_1 b_2}$ denotes p_k with $k = 4b_0 + 2b_1 + b_2$. If $p_{b_0 b_1 0} + p_{b_0 b_1 1} = 0$ (zero subtree mass), set $\theta_{2|b_0 b_1} = 0$ by convention.

3.2 Walsh–Hadamard transform for the UCRy block

The four level-2 angles are decomposed into the ancilla-free ladder representation via the Walsh–Hadamard-type transform [2]. Let $\alpha_j = \theta_{2|j}/2$ for $j \in \{00, 01, 10, 11\}$ (ordered as binary $j = 0, 1, 2, 3$). The ladder angles are:

$$\begin{pmatrix} \phi_0 \\ \phi_1 \\ \phi_2 \\ \phi_3 \end{pmatrix} = \frac{1}{4} \begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & -1 & 1 & -1 \\ 1 & 1 & -1 & -1 \\ 1 & -1 & -1 & 1 \end{pmatrix} \begin{pmatrix} \alpha_{00} \\ \alpha_{01} \\ \alpha_{10} \\ \alpha_{11} \end{pmatrix}. \tag{6}$$

The full rotation angles commanded to the hardware are $2\phi_i$ (in degrees).

3.3 Reference implementation

Listing 2: Parametric GR angle computation for any 3-qubit distribution.

```python
import numpy as np

def gr_angles(p: np.ndarray) -> dict:
    """
    Compute all Grover-Rudolph angles for a 3-qubit
```

```
    probability vector p of length 8.
    Returns angles in degrees.
    """
    assert len(p) == 8 and np.isclose(p.sum(), 1.0)

    sL  = p[0]+p[1]+p[2]+p[3]
    sR  = 1.0 - sL
    sLL = p[0]+p[1]; sLR = p[2]+p[3]
    sRL = p[4]+p[5]; sRR = p[6]+p[7]

    def safe_arccos(num, den):
        if den < 1e-15:
            return 0.0
        return 2 * np.degrees(np.arccos(np.sqrt(
            np.clip(num/den, 0.0, 1.0))))

    theta0   = safe_arccos(sL, 1.0)
    theta1_0 = safe_arccos(sLL, sL)
    theta1_1 = safe_arccos(sRL, sR)
    theta2   = {
        '00': safe_arccos(p[0], sLL),
        '01': safe_arccos(p[2], sLR),
        '10': safe_arccos(p[4], sRL),
        '11': safe_arccos(p[6], sRR),
    }

    # Walsh-Hadamard transform for UCRy ladder
    alpha = np.array([theta2['00'], theta2['01'],
                      theta2['10'], theta2['11']]) / 2.0
    H = np.array([[1,1,1,1],[1,-1,1,-1],
                  [1,1,-1,-1],[1,-1,-1,1]]) / 4.0
    phi = H @ alpha  # ladder half-angles in degrees

    return {
        'theta0':   theta0,
        'theta1_0': theta1_0,
        'theta1_1': theta1_1,
        'theta2':   theta2,
        'phi':      phi,          # ladder half-angles
        'ladder_angles': 2*phi,   # full angles for hardware
    }
```

Verification step (mandatory for every distribution). After computing angles, run the ideal simulator and confirm $\mathrm{TV}(q_{\mathrm{sim}}, p) \leq 10^{-10}$ and $\mathrm{Fid}(q_{\mathrm{sim}}, p) = 1.000$. Any failure indicates a formula or implementation error; do not proceed to hardware until resolved.

4 Circuit structure and ladder variants

4.1 Diagnostic stages

The three nested diagnostic stages $L0$, $L01$, FULL are preserved from the first campaign and applied identically to every distribution in the suite. Their definitions in terms of the dyadic tree levels are unchanged: $L0$ applies only θ_0; $L01$ applies θ_0 and the level-1 angles; FULL applies the complete construction.

The gate counts depend only on the stage and ladder, not on the specific distribution, since the circuit topology is fixed by the UCRy decomposition:

Table 2: Gate counts per stage and ladder (identical for all distributions D0–D6).

Stage	Total gates	R_y	CNOT	X
$L0$	3	3	0	0
$L01$	13	7	4	2
FULL-A	21	11	8	2
FULL-B	21	11	8	2

4.2 Ladder variants A and B

The two FULL circuits differ only in the Gray-code ordering of the two-control UCRy block on q_2, as in the first campaign:

- **Ladder A.** UCRy control sequence: (q_1, q_0, q_0, q_0). Sub-rotation order: $R_y(2\phi_0)$, CNOT(q_1), $R_y(2\phi_3)$, CNOT(q_0), $R_y(2\phi_2)$, CNOT(q_0), $R_y(2\phi_1)$, CNOT(q_0).
- **Ladder B.** UCRy control sequence: (q_1, q_1, q_0, q_0). Sub-rotation order: $R_y(2\phi_0)$, CNOT(q_1), $R_y(2\phi_1)$, CNOT(q_1), $R_y(2\phi_2)$, CNOT(q_0), $R_y(2\phi_3)$, CNOT(q_0).

Note that the ladder angles ϕ_i now depend on the target distribution through equation (6), so ladders A and B produce physically different pulse schedules for each distribution in the suite. The unitary equivalence is preserved in all cases.

5 Automated execution pipeline

5.1 Campaign structure

The full campaign is organized as follows:

- **Distributions:** 7 (D0–D6).
- **Stages per distribution:** 3 ($L0$, $L01$, FULL).
- **Ladder variants (FULL only):** 2 (A, B).
- **Runs per (distribution, stage, ladder):** 25.
- **Total runs:** $7 \times (50 + 50 + 25 + 25) = 1050$.

Compared with the first campaign (150 runs, 1 distribution), the extended campaign is a factor-7 scale-up in total hardware time, motivating the automated pipeline described below.

5.2 Pipeline architecture

The pipeline is organised as four independent modules that are called sequentially for each (distribution, stage, ladder) triplet:

1. **Angle engine.** Loads the target distribution, calls `gr_angles()`, and returns the full angle dictionary.
2. **Circuit compiler.** Takes the angle dictionary and stage/ladder specification, instantiates the gate sequence, and serializes it to the platform's JSON circuit format.
3. **Job executor.** Submits the compiled circuit to the Triangulum SDK, polls until completion, retrieves raw counts, and appends the result to the JSONL log.

4. **Run validator.** Checks that the returned count sum is within 1% of the nominal shot number and that no error status was returned; marks the run as `failed` if either condition is violated.

5.3 Orchestrator

Listing 3: Campaign orchestrator: iterates over all distributions, stages, and ladders.

```
import json
import itertools

DISTRIBUTIONS = {
    'D0': np.full(8, 0.125),
    'D1': np.array([0.05,0.10,0.15,0.20,
                    0.20,0.15,0.10,0.05]),
    'D2': np.array([0.35,0.25,0.15,0.10,
                    0.07,0.04,0.03,0.01]),
    'D3': np.array([0.02,0.03,0.05,0.40,
                    0.40,0.05,0.03,0.02]),
    'D4': np.array([0.02,0.05,0.12,0.31,
                    0.31,0.12,0.05,0.02]),
    'D5': load_from_config('D5'),   # from campaign_distributions.json
    'D6': load_from_config('D6'),
}
STAGES   = ['L0', 'L01', 'FULL']
LADDERS  = {'L0': ['A'], 'L01': ['A'], 'FULL': ['A', 'B']}
N_RUNS   = 25   # runs per (dist, stage, ladder)
LOGFILE  = 'campaign_v2_runs.jsonl'

def run_campaign():
    run_index = 0
    for dist_id, p in DISTRIBUTIONS.items():
        angles = gr_angles(p)
        verify_simulation(p, angles)       # mandatory check
        for stage in STAGES:
            for ladder in LADDERS[stage]:
                circuit = compile_circuit(angles, stage, ladder)
                for rep in range(1, N_RUNS + 1):
                    run_index += 1
                    record = execute_and_log(
                        circuit=circuit,
                        dist_id=dist_id,
                        p_target=p,
                        stage=stage,
                        ladder=ladder,
                        repeat=rep,
                        run_index=run_index,
                    )
                    log_run(record, LOGFILE)

if __name__ == '__main__':
    run_campaign()
```

5.4 Execution order and drift mitigation

To mitigate systematic hardware drift across the campaign, runs are interleaved across distributions rather than executing all runs for one distribution before moving to the next. The

recommended block structure is:

1. For each stage ($L0$, $L01$, FULL), execute one block of 5 runs per distribution per ladder, cycling through all 7 distributions.
2. Repeat the full cycle 5 times to accumulate 25 runs per group.
3. After every 100 runs, perform a bare-state fidelity check; suspend if $P(|000\rangle) < 0.97$.

This interleaved structure ensures that any slow drift in the device affects all distributions approximately equally, so inter-distribution comparisons remain valid.

5.5 Run log schema

Each run is logged as a single JSON object appended to `campaign_v2_runs.jsonl`. In addition to the fields from Protocol v1.0, the following fields are added:

Field	Type	Description
`dist_id`	string	Distribution identifier (D0–D6)
`dist_type`	string	Human-readable type (e.g. `uniform`)
`p_target`	array	Target probability vector (8 floats)
`shannon_entropy`	float	$H_{\mathrm{S}}(p)$ in bits
`contrast`	float	$\max_k p_k / \min_{k:p_k>0} p_k$
`angles`	object	Full angle dictionary from `gr_angles()`
`run_index`	integer	Global run index across the full campaign

6 Data normalization, readout calibration, and metrics

6.1 Probability normalization and marginals

Identical to Protocol v1.0. Raw counts are normalized to probability vectors via equation (7); per-qubit marginals are computed via equation (8).

$$q_k = \frac{c_k}{\sum_{j=0}^{7} c_j}, \quad k = 0, \ldots, 7. \tag{7}$$

$$P(q_j = b) = \sum_{\substack{k=0 \\ b_j(k)=b}}^{7} q_k. \tag{8}$$

6.2 Readout calibration

The $L0$ stage provides a per-qubit assignment error estimate for each distribution run. Since the $L0$ state is identical for all distributions (only q_0 is active), the readout error estimates from D0–D6 can be pooled to obtain a more precise characterisation of the device's measurement bias across the campaign:

$$\varepsilon_{0\to1}(Q_j) = 1 - \frac{1}{7 \cdot 50} \sum_{\text{dist}} \sum_{\text{run}} P_{\text{exp}}(q_j = 0)\Big|_{L0,\text{dist}}, \quad j \in \{1, 2\}. \tag{9}$$

This pooled estimate uses $7 \times 50 = 350$ $L0$ runs and gives a substantially tighter characterisation of readout error than the 50 runs available in the first campaign.

6.3 Run-level metrics

For each run, compute the six metrics as in Protocol v1.0:

$$\mathrm{TV}(p,q) = \tfrac{1}{2}\sum_{k=0}^{7} |p_k - q_k|, \tag{10}$$

$$\|p-q\|_2 = \left(\textstyle\sum_{k=0}^{7}(p_k-q_k)^2\right)^{1/2}, \tag{11}$$

$$\mathrm{Fid}(p,q) = \left(\textstyle\sum_{k=0}^{7}\sqrt{p_k q_k}\right)^2. \tag{12}$$

Compute against both the simulator output and the target: `tv_vs_sim`, `l2_vs_sim`, `fidelity_vs_sim`, `tv_vs_target`, `l2_vs_target`, `fidelity_vs_target`.

6.4 Output data artifacts

The campaign produces the following artifacts:

1. `campaign_v2_runs.jsonl` — complete per-run records for all 1050 runs.
2. `runs_flat_v2.csv` — one row per run, all metric and probability fields, with additional columns `dist_id`, `shannon_entropy`, `contrast`.
3. `summary_by_dist_stage.csv` — one row per (dist_id, stage, ladder) group ($7 \times 4 = 28$ rows), with means and standard deviations of all metrics.
4. `summary_by_dist_FULL.csv` — one row per distribution at the FULL stage only ($7\times 2 = 14$ rows), used for the cross-distribution characterisation analysis.
5. `circuits_flat_v2.csv` — one row per (dist_id, stage, ladder) combination (28 rows), with angle values, gate counts, and circuit payloads.

7 Cross-distribution analysis and characterisation outputs

7.1 Primary characterisation plot

The central output of the extended campaign is a *performance map* of the Triangulum: a plot of mean FULL-stage classical fidelity vs. Shannon entropy $H_{\mathrm{S}}(p)$, with one point per distribution and error bars equal to the standard deviation over 25 runs per ladder. Ladders A and B are shown separately. This plot provides a quantitative answer to the question of how hardware performance varies with target complexity.

Expected qualitative behaviour. Based on the first campaign and the structure of the circuits, we anticipate a monotone-increasing trend (higher entropy $\Rightarrow$ less demanding UCRy block $\Rightarrow$ higher fidelity), with D0 (uniform) achieving the highest fidelity and D3 or D5 (bimodal / sparse) the lowest. Deviations from this trend are scientifically informative and should be reported and discussed.

7.2 Stage-wise degradation per distribution

For each distribution, compute the stage-wise TV distances and fidelities as in the first campaign, and report the degradation trajectory ($L0 \to L01 \to$ FULL). A distribution for which the $L0 \to L01$ step is larger than the $L01 \to$ FULL step (in TV units) exhibits primarily level-1 control error; the reverse pattern implicates the UCRy block.

7.3 Ladder A vs. B comparison per distribution

Apply the two-sided Mann–Whitney U test for ladders A vs. B at the FULL stage, independently for each distribution and each metric. Summarise results in a 7×3 table (distributions $\times$ metrics) of p-values. Test for a consistent pattern: if ladder ordering matters more for concentrated distributions (D3, D5) than for smooth ones (D0, D1), this suggests that near-zero controlled rotations are particularly sensitive to Gray-code ordering.

7.4 Readout error characterisation across distributions

Plot $\Delta\,\mathrm{TV} = \mathrm{TV}(q_{\mathrm{exp}}, p) - \mathrm{TV}(q_{\mathrm{exp}}, q_{\mathrm{sim}})$ at stage $L0$ as a function of distribution. Since the $L0$ circuit is identical for all distributions, any variation in $\Delta\,\mathrm{TV}$ across distributions is attributable to run-to-run hardware variability, not to the target itself. This provides a direct estimate of the precision of the readout error characterisation.

7.5 Regression: fidelity vs. circuit properties

Fit a simple linear model:

$$\mathrm{Fid}_{\mathrm{FULL}} \approx \beta_0 + \beta_1 H_{\mathrm{S}}(p) + \beta_2 \log \mathrm{Contrast}(p) + \beta_3 \max_j |\theta_{2|j} - 90^\circ| + \varepsilon. \tag{13}$$

Report the estimated coefficients, R^2, and the 95% confidence intervals. This model, while approximate, quantifies the relative contribution of entropy, contrast, and UCRy angle extremity to hardware-level fidelity loss.

Listing 4: Cross-distribution regression.

```python
import numpy as np
from scipy import stats

def characterisation_regression(summary_df):
    """
    summary_df: one row per distribution at FULL stage.
    Columns: shannon_entropy, log_contrast,
             max_ucry_deviation, fidelity_vs_target_mean
    """
    X = summary_df[['shannon_entropy',
                    'log_contrast',
                    'max_ucry_deviation']].values
    X = np.column_stack([np.ones(len(X)), X])
    y = summary_df['fidelity_vs_target_mean'].values

    beta, res, rank, sv = np.linalg.lstsq(X, y,
                                          rcond=None)
    y_hat = X @ beta
    ss_res = np.sum((y - y_hat)**2)
    ss_tot = np.sum((y - y.mean())**2)
    r2 = 1 - ss_res / ss_tot

    return {'beta': beta, 'R2': r2}
```

Data and code availability

All data artifacts and the full automation pipeline will be released at:

https://github.com/afalco/grover-rudolph-practical-implementation

The repository will include a `campaigns/v2/` subdirectory with the orchestrator, angle engine, and post-processing scripts, as well as the fixed campaign configuration file `campaign_distributions.json`.

A Distribution checksums

After generating D5 and D6 via the procedure in Section 2.4, verify the following SHA-256 checksums of the JSON-serialised vectors (compact format, 15 decimal places):

```
D5: [to be completed after generation]
D6: [to be completed after generation]
```

These values will be committed to the repository before the campaign begins and serve as the reference for all participants.

B Expected results: first campaign anchor (D1)

For distribution D1 (the reference from Protocol v1.0), the extended campaign should reproduce the following results within $\pm 2\sigma$ of the first campaign values. Systematic deviations indicate hardware drift between campaigns.

Table 4: First-campaign reference values for D1 (25 runs per ladder per stage).

Stage	Ladder	TV vs target	ℓ^2 vs target	Fidelity vs target
$L0$	A	0.522 ± 0.039	0.455 ± 0.031	0.657 ± 0.043
	B	0.530 ± 0.041	0.465 ± 0.032	0.647 ± 0.043
$L01$	A	0.352 ± 0.038	0.299 ± 0.030	0.804 ± 0.036
	B	0.345 ± 0.046	0.290 ± 0.035	0.817 ± 0.048
FULL	A	0.234 ± 0.047	0.195 ± 0.037	0.916 ± 0.033
	B	0.232 ± 0.059	0.201 ± 0.047	0.909 ± 0.038

References

[1] Grover, L., Rudolph, T.: Creating superpositions that correspond to efficiently integrable probability distributions. arXiv:quant-ph/0208112 (2002)

[2] Falcó, A., Falcó-Pomares, D., Matthies, H.G.: A rigorous and self-contained proof of the Grover–Rudolph state preparation algorithm. arXiv:2601.17930 (2026). Submitted to AIMS Mathematics

[3] Feng, G., Hou, S.-Y., Zou, H., et al.: SpinQ Triangulum: a commercial three-qubit desktop quantum computer. arXiv:2202.02983 (2022)

[4] Falcó, A., Falcó-Pomares, D., Latorre, J.-C., et al.: Supplementary Material: Experimental Protocol for the First Grover–Rudolph Campaign on the SpinQ Triangulum Platform. Version 1.0 (2026). Available at the companion repository.